

Random Points (<https://mortada.net/>)

Search

» Software

- About (</pages/about.html>)
- Math (<https://mortada.net/category/math.html>)
- Misc (<https://mortada.net/category/misc.html>)
- Photography (<https://mortada.net/category/photography.html>)
- Software (<https://mortada.net/category/software.html>)

Simple Proof for Kraft's Inequality

Wed 04 March 2015

In graduate school I came up with an original proof for [Kraft's inequality](http://en.wikipedia.org/wiki/Kraft%27s_inequality) (http://en.wikipedia.org/wiki/Kraft%27s_inequality). The proofs I could find in textbooks tend to be more complicated and less intuitive to me, and so I'd like to share my proof here.

Prefix Codes

First it is important to understand the concept of [prefix codes](http://en.wikipedia.org/wiki/Prefix_code) (http://en.wikipedia.org/wiki/Prefix_code). Prefix codes have nice properties and are widely used in applications such as JPEG encoding for images and MP3 for music. A prefix code is such that no codeword is a prefix of any other codewords. For instance, suppose we'd like to encode the letters $\{a, b, c, d\}$ in binaries, i.e. using $\{0, 1\}$. Then one possible coding scheme is as follows:

$$\begin{cases} a \rightarrow 0 \\ b \rightarrow 01 \\ c \rightarrow 011 \\ d \rightarrow 0111 \end{cases}$$

This is uniquely decodable but it is not a prefix code because the codeword for a is a prefix for the codeword for b . This means that we can not instantaneously decode a without waiting for the next bit of data (to determine whether it is actually a or just the first half of b .)

Alternatively, we can encode using a prefix code as follows:

$$\begin{cases} a \rightarrow 0 \\ b \rightarrow 10 \\ c \rightarrow 110 \\ d \rightarrow 111 \end{cases}$$

As you can see no codeword is a prefix of another codeword. Naturally we'd like to know how to construct prefix codes efficiently, and what are the fundamental constraints on the lengths of the codewords.

Kraft's Inequality

Kraft's inequality states that given a list of positive integers $(n_1, n_2, \dots, n_r)$, there exists a prefix code with a set of codewords $(\sigma_1, \sigma_2, \dots, \sigma_r)$ where the length of each codeword $|\sigma_i| = n_i, \forall i$, if and only if

$$\sum_{i=1}^r s^{-n_i} \leq 1$$

where s is the size of the alphabet S .

In our previous example, the alphabet is simply the binary digits $S = \{0, 1\}$, and therefore the size of the alphabet $s = 2$. We can easily check that indeed the inequality holds:

$$2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} \leq 1$$

Simple Proof

First of all without loss of generality we can order n_i so that

$$n_1 \leq n_2 \leq \dots \leq n_r$$

Now we try to construct a prefix code in increasing order, i.e., $i = 1, 2, \dots, r$. There exists a prefix code if and only if at each step j , there is at least one codeword to choose that does not contain any of the previous $j - 1$ codewords as a prefix. In other words, the codeword σ_j must not contain any of the previous codewords $(\sigma_1, \sigma_2, \dots, \sigma_{j-1})$ as a prefix. For σ_j , there are s^{n_j} combination of codewords we could choose from if there was no prefix constraints. But due to the codeword at a prior step $k < j$, $s^{n_j - n_k}$ codewords are now forbidden because they contain σ_k as a prefix. Furthermore, the sets of forbidden codewords due to different prior steps are exclusive. Since if any two prior steps forbid the same codeword at step j , that would imply the smaller of the codewords in the two prior steps is a prefix of the other one, which contradicts the prefix assumption.

Therefore we know the total number of forbidden codewords at step j is

$$\sum_{i=1}^{j-1} s^{n_j - n_i}$$

There exists a prefix code if and only if we have a codeword to choose at every step, namely,

$$s^{n_j} > \sum_{i=1}^{j-1} s^{n_j - n_i}, \forall j = 2, 3, \dots, r$$

Since every term above is an integer, this is equivalent to

$$s^{n_j} \geq \sum_{i=1}^{j-1} s^{n_j - n_i} + 1 = \sum_{i=1}^j s^{n_j - n_i}, \forall j = 1, 2, \dots, r$$

Now divide by s^{n_j} on both sides we get

$$1 \geq \sum_{i=1}^j s^{-n_i}, \forall j = 1, 2, \dots, r$$

Now since every term above is non-negative, this is equivalent to Kraft's inequality

$$\sum_{i=1}^r s^{-n_i} \leq 1$$

Notice that every argument in the proof goes both ways. Thus this proves that Kraft's inequality is both a necessary and sufficient condition for the existence of a prefix code.

McMillian Inequality

Interestingly, Kraft's inequality can be shown to hold for all uniquely decodable codes, not just prefix codes. This result is also known as the Kraft-McMillian inequality. To see that, consider a sequence of k codewords from any uniquely decodable code. For instance, for a uniquely decodable code

$$\begin{cases} a \rightarrow 0 \\ b \rightarrow 01 \\ c \rightarrow 011 \end{cases}$$

all possible sequences of k codewords for $k = 2$ would be

$$\begin{cases} aa \rightarrow 00 \\ ab \rightarrow 001 \\ ac \rightarrow 0011 \\ ba \rightarrow 010 \\ bb \rightarrow 0101 \\ bc \rightarrow 01011 \\ ca \rightarrow 0110 \\ cb \rightarrow 01101 \\ cc \rightarrow 011011 \end{cases}$$

Now consider the following expression

$$\left(\sum_{i=1}^r s^{-n_i}\right)^k$$

This is equivalent to

$$\sum_{j=k \cdot n_1}^{k \cdot n_r} C_j s^{-j}$$

where C_j is the number of combinations of k codewords that have a total length of j . We can see that j must be at least $k \cdot n_1$ and at most $k \cdot n_r$. Since there is only s^j number of unique sequences with a length of j , in order for this code to be uniquely decodable, we must have

$$C_j \leq s^j$$

Therefore we now have

$$\begin{aligned} \left(\sum_{i=1}^r s^{-n_i}\right)^k &= \sum_{j=k \cdot n_1}^{k \cdot n_r} C_j s^{-j} \\ &\leq \sum_{j=k \cdot n_1}^{k \cdot n_r} s^j s^{-j} \\ &= \sum_{j=k \cdot n_1}^{k \cdot n_r} 1 \end{aligned}$$

Shannon Entropy

Kraft's inequality can be used to show that any uniquely decodable code must have an expected length larger than or equal to the [Shannon Entropy](http://en.wiktionary.org/wiki/Shannon_entropy). If the codeword σ_i occurs with probability p_i , then the Shannon Entropy is given by

$$\sum_{i=1}^r p_i \log_s \frac{1}{p_i}$$

To see that the expected code length has to be bound by the Shannon entropy, consider the difference between the Shannon entropy and the expected length of the code:

$$\begin{aligned} \sum_{i=1}^r p_i \log_s \frac{1}{p_i} - \sum_{i=1}^r p_i n_i &= \sum_{i=1}^r p_i \log_s \frac{s^{-n_i}}{p_i} \\ &\leq \log_s \left(\sum_{i=1}^r p_i \frac{s^{-n_i}}{p_i} \right) \\ &= \log_s \left(\sum_{i=1}^r s^{-n_i} \right) \\ &\leq \log_s 1 = 0 \end{aligned}$$

where we simply use the fact that $\log_s$ is a concave function and apply Kraft's inequality.

Posted by Mortada Mehyar Wed 04 March 2015 [math](https://mortada.net/category/math.html) (<https://mortada.net/category/math.html>) [information theory](https://mortada.net/tag/information-theory.html) (<https://mortada.net/tag/information-theory.html>), [coding theory](https://mortada.net/tag/coding-theory.html) (<https://mortada.net/tag/coding-theory.html>), [prefix code](https://mortada.net/tag/prefix-code.html) (<https://mortada.net/tag/prefix-code.html>), [Kraft's inequality](https://mortada.net/tag/krafts-inequality.html) (<https://mortada.net/tag/krafts-inequality.html>), [Kraft-McMillan inequality](https://mortada.net/tag/kraft-mcmillan-inequality.html) (<https://mortada.net/tag/kraft-mcmillan-inequality.html>), [Shannon entropy](https://mortada.net/tag/shannon-entropy.html) (<https://mortada.net/tag/shannon-entropy.html>), [uniquely decodable](https://mortada.net/tag/uniquely-decodable.html) (<https://mortada.net/tag/uniquely-decodable.html>)

Post Like Share One person likes this. Sign Up to see what your friends like.

Comments

ALSO ON MORTADA.NET

Tips for Running TensorFlow with ...

8 years ago • 6 comments

In this blog post I will discuss how to get TensorFlow working on ...

3 Easy Steps to Set Up Pyspark — ...

7 years ago • 1 comment

Starting with Spark 2.2, it is now super easy to set up pyspark. Download Spark ...

Can Integer Operations ...

10 years ago • 6 comments

If the number of bits used is fixed, the range of integers that can be represented ...

Computing Sample Variance: Why ...

10 years ago • 1 comment

In statistics we know that the mean and variance of a population μ and σ^2 are ...

The Traveling Tesla Salesman — ...

10 years ago • 22 comments

Solving the TSP¶ There are numerous heuristics and approximate solutions for ...

1 Comment

Login

G

Join the discussion...

LOG IN WITH

OR SIGN UP WITH DISQS ?

Name

♥ 1 Share [Best](#) [Newest](#) [Oldest](#)

 **jason** 9 years ago edited

Thanks for sharing the proof of McMillan Inequality. Does it make a assumption that the minimum length of codeword is 1, so minimum length of combinations is n (so k = n)

o o Reply Share

[Subscribe](#) [Privacy](#) [Do Not Sell My Data](#)

Recent Posts

- [Left Tesla](https://mortada.net/left-tesla.html) (<https://mortada.net/left-tesla.html>)
- [3 Easy Steps to Set Up Pyspark](https://mortada.net/3-easy-steps-to-set-up-pyspark.html) (<https://mortada.net/3-easy-steps-to-set-up-pyspark.html>)
- [Tips for Running TensorFlow with GPU Support on AWS](https://mortada.net/tips-for-running-tensorflow-with-gpu-support-on-aws.html) (<https://mortada.net/tips-for-running-tensorflow-with-gpu-support-on-aws.html>)
- [Easily Profile Python Code in Jupyter](https://mortada.net/easily-profile-python-code-in-jupyter.html) (<https://mortada.net/easily-profile-python-code-in-jupyter.html>)
- [Now Working At Tesla](https://mortada.net/now-working-at-tesla.html) (<https://mortada.net/now-working-at-tesla.html>)

Categories

- [math \(https://mortada.net/category/math.html\)](https://mortada.net/category/math.html)
- [misc \(https://mortada.net/category/misc.html\)](https://mortada.net/category/misc.html)
- [photography \(https://mortada.net/category/photography.html\)](https://mortada.net/category/photography.html)
- [software \(https://mortada.net/category/software.html\)](https://mortada.net/category/software.html)

Tags

[tesla \(https://mortada.net/tag/tesla.html\)](https://mortada.net/tag/tesla.html), [conda \(https://mortada.net/tag/conda.html\)](https://mortada.net/tag/conda.html), [pip \(https://mortada.net/tag/pip.html\)](https://mortada.net/tag/pip.html), [pyspark \(https://mortada.net/tag/pyspark.html\)](https://mortada.net/tag/pyspark.html), [spark \(https://mortada.net/tag/spark.html\)](https://mortada.net/tag/spark.html), [tensorflow \(https://mortada.net/tag/tensorflow.html\)](https://mortada.net/tag/tensorflow.html), [AWS \(https://mortada.net/tag/aws.html\)](https://mortada.net/tag/aws.html), [python \(https://mortada.net/tag/python.html\)](https://mortada.net/tag/python.html), [GPU \(https://mortada.net/tag/gpu.html\)](https://mortada.net/tag/gpu.html), [CUDA \(https://mortada.net/tag/cuda.html\)](https://mortada.net/tag/cuda.html), [cuDNN \(https://mortada.net/tag/cudnn.html\)](https://mortada.net/tag/cudnn.html), [numpy \(https://mortada.net/tag/numpy.html\)](https://mortada.net/tag/numpy.html), [pandas \(https://mortada.net/tag/pandas.html\)](https://mortada.net/tag/pandas.html), [jupyter notebook \(https://mortada.net/tag/jupyter-notebook.html\)](https://mortada.net/tag/jupyter-notebook.html), [vectorization \(https://mortada.net/tag/vectorization.html\)](https://mortada.net/tag/vectorization.html), [optimization \(https://mortada.net/tag/optimization.html\)](https://mortada.net/tag/optimization.html), [line profiler \(https://mortada.net/tag/line-profiler.html\)](https://mortada.net/tag/line-profiler.html), [traveling salesman problem \(https://mortada.net/tag/traveling-salesman-problem.html\)](https://mortada.net/tag/traveling-salesman-problem.html), [algorithms \(https://mortada.net/tag/algorithms.html\)](https://mortada.net/tag/algorithms.html), [superchargers \(https://mortada.net/tag/superchargers.html\)](https://mortada.net/tag/superchargers.html), [TSP \(https://mortada.net/tag/tsp.html\)](https://mortada.net/tag/tsp.html), [concorde \(https://mortada.net/tag/concorde.html\)](https://mortada.net/tag/concorde.html), [complexity \(https://mortada.net/tag/complexity.html\)](https://mortada.net/tag/complexity.html), [statistics \(https://mortada.net/tag/statistics.html\)](https://mortada.net/tag/statistics.html), [probability \(https://mortada.net/tag/probability.html\)](https://mortada.net/tag/probability.html), [unbiased variance estimator \(https://mortada.net/tag/unbiased-variance-estimator.html\)](https://mortada.net/tag/unbiased-variance-estimator.html), [standard deviation \(https://mortada.net/tag/standard-deviation.html\)](https://mortada.net/tag/standard-deviation.html), [population mean \(https://mortada.net/tag/population-mean.html\)](https://mortada.net/tag/population-mean.html), [population variance \(https://mortada.net/tag/population-variance.html\)](https://mortada.net/tag/population-variance.html), [sample mean \(https://mortada.net/tag/sample-mean.html\)](https://mortada.net/tag/sample-mean.html), [sample variance \(https://mortada.net/tag/sample-variance.html\)](https://mortada.net/tag/sample-variance.html), [biased estimator \(https://mortada.net/tag/biased-estimator.html\)](https://mortada.net/tag/biased-estimator.html), [pydata \(https://mortada.net/tag/pydata.html\)](https://mortada.net/tag/pydata.html), [numerical precision \(https://mortada.net/tag/numerical-precision.html\)](https://mortada.net/tag/numerical-precision.html), [arbitrary precision integers \(https://mortada.net/tag/arbitrary-precision-integers.html\)](https://mortada.net/tag/arbitrary-precision-integers.html), [integer overflow \(https://mortada.net/tag/integer-overflow.html\)](https://mortada.net/tag/integer-overflow.html), [information theory \(https://mortada.net/tag/information-theory.html\)](https://mortada.net/tag/information-theory.html), [coding theory \(https://mortada.net/tag/coding-theory.html\)](https://mortada.net/tag/coding-theory.html), [prefix code \(https://mortada.net/tag/prefix-code.html\)](https://mortada.net/tag/prefix-code.html), [Kraft's inequality \(https://mortada.net/tag/krafts-inequality.html\)](https://mortada.net/tag/krafts-inequality.html), [Kraft-McMillan inequality \(https://mortada.net/tag/kraft-mcmillan-inequality.html\)](https://mortada.net/tag/kraft-mcmillan-inequality.html), [Shannon entropy \(https://mortada.net/tag/shannon-entropy.html\)](https://mortada.net/tag/shannon-entropy.html), [uniquely decodable \(https://mortada.net/tag/uniquely-decodable.html\)](https://mortada.net/tag/uniquely-decodable.html), [Fibonacci numbers \(https://mortada.net/tag/fibonacci-numbers.html\)](https://mortada.net/tag/fibonacci-numbers.html), [tail recursion optimization \(https://mortada.net/tag/tail-recursion-optimization.html\)](https://mortada.net/tag/tail-recursion-optimization.html), [economic data \(https://mortada.net/tag/economic-data.html\)](https://mortada.net/tag/economic-data.html), [FRED \(https://mortada.net/tag/fred.html\)](https://mortada.net/tag/fred.html)

Follow [@mortada](https://twitter.com/mortada)

Copyright © 2012–2022 Mortada Mehyar — Powered by [Pelican \(http://getpelican.com\)](http://getpelican.com)